

DIP HW4_2

313512009 施丞蔚

1. Please use FFT and design a frequency filter to cancel the sinusoidal noise of the assigned image, 'astronaut-interference.tif', and print out the source code and the processed image? (40)

```
# -*- coding: utf-8 -*-

import argparse
import os
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

def read_gray(path):
    if not os.path.exists(path):
        raise FileNotFoundError(f"找不到輸入影像: {path}")
    return np.array(Image.open(path).convert("L"), dtype=np.float32)

def normalize_u8(x):
    x = (x - x.min()) / (x.max() - x.min() + 1e-8) * 255.0
    return x.astype(np.uint8)

def build_rect_mask(shape, peaks_xy, half):
    H, W = shape
    cy, cx = H//2, W//2
    mask = np.ones(shape, dtype=np.float32)
    for (x, y) in peaks_xy:
        for dy in range(-half, half+1):
            for dx in range(-half, half+1):
                yy, xx = y+dy, x+dx
                if 0 <= yy < H and 0 <= xx < W:
                    mask[yy, xx] = 0.0
                ys = (2*cy - yy) % H
                xs = (2*cx - xx) % W
                mask[ys, xs] = 0.0
    return mask

def build_gauss_mask(shape, peaks_xy, sigma):
    H, W = shape
    yy = np.arange(H)[None, :]
    xx = np.arange(W)[None, :]
```

```

cy, cx = H//2, W//2
def notch(x0,y0):
    return 1.0 - np.exp(-((yy-y0)**2 + (xx-x0)**2)/(2*sigma*sigma))
Hf = np.ones(shape, dtype=np.float32)
for (x0, y0) in peaks_xy:
    Hf *= notch(x0, y0)
    y1 = (2*cy - y0) % H
    x1 = (2*cx - x0) % W
    Hf *= notch(x1, y1)
return Hf

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--input", default="astronaut-interference.tif")
    ap.add_argument("--mode", choices=["rect", "gauss"], default="gauss")
    ap.add_argument("--half", type=int, default=8, help="rect mask
half-width (pixels)")
    ap.add_argument("--sigma", type=float, default=7.0, help="gaussian
notch sigma")
    ap.add_argument("--no-show", action="store_true", help="只存檔，不跳視
窗")
    args = ap.parse_args()

    img = read_gray(args.input)
    H, W = img.shape

    # 2D FFT + center
    F = np.fft.fft2(img)
    F0 = np.fft.fftshift(F)
    amp = np.abs(F0)

    # 兩個主峰位置 (搭配共軛點)
    peaks_xy = [(475, 387), (525, 437)] # (x=col, y=row)

    # 選擇濾波器
    if args.mode == "rect":
        Hf = build_rect_mask((H, W), peaks_xy, half=args.half)
        out_path = "astronaut_interference_filtered_manual_rect.png"
    else:
        Hf = build_gauss_mask((H, W), peaks_xy, sigma=args.sigma)
        out_path = "astronaut_interference_filtered_gauss_notch.png"

    # 濾波與 IFFT

```

```

Ff    = F0 * Hf
img2 = np.fft.ifft2(np.fft.ifftshift(Ff)).real
img2_u8 = normalize_u8(img2)
Image.fromarray(img2_u8).save(out_path)
print(f"[Saved] result: {out_path}")

# ===== 中間過程也另存檔, 並可視化 =====

out_dir = "resault_q1"
os.makedirs(out_dir, exist_ok=True)

# 1) 原始輸入影像
plt.figure()
plt.title("Original")
plt.imshow(img.astype(np.uint8), cmap="gray")
plt.axis("off")
plt.tight_layout()
plt.savefig(os.path.join(out_dir, "original.png"), dpi=300,
bbox_inches="tight")
print("[Saved] original.png")

# 2) FFT amplitude 圖
plt.figure()
plt.title("fft amp")
fft_amp_vis = np.log1p(amp) / np.max(np.log1p(amp))
plt.imshow(fft_amp_vis, cmap="gray")
plt.axis("on")
plt.tight_layout()
plt.savefig(os.path.join(out_dir, "fft_amp.png"), dpi=300,
bbox_inches="tight")
print("[Saved] fft_amp.png")

# 3) Mask 之後的頻譜圖
plt.figure()
plt.title("mask result")
masked_vis = np.log1p(np.abs(Ff)) / np.max(np.log1p(np.abs(Ff)))
plt.imshow(masked_vis, cmap="gray")
plt.axis("on")
plt.tight_layout()
plt.savefig(os.path.join(out_dir, "mask_result.png"), dpi=300,
bbox_inches="tight")
print("[Saved] mask_result.png")

```

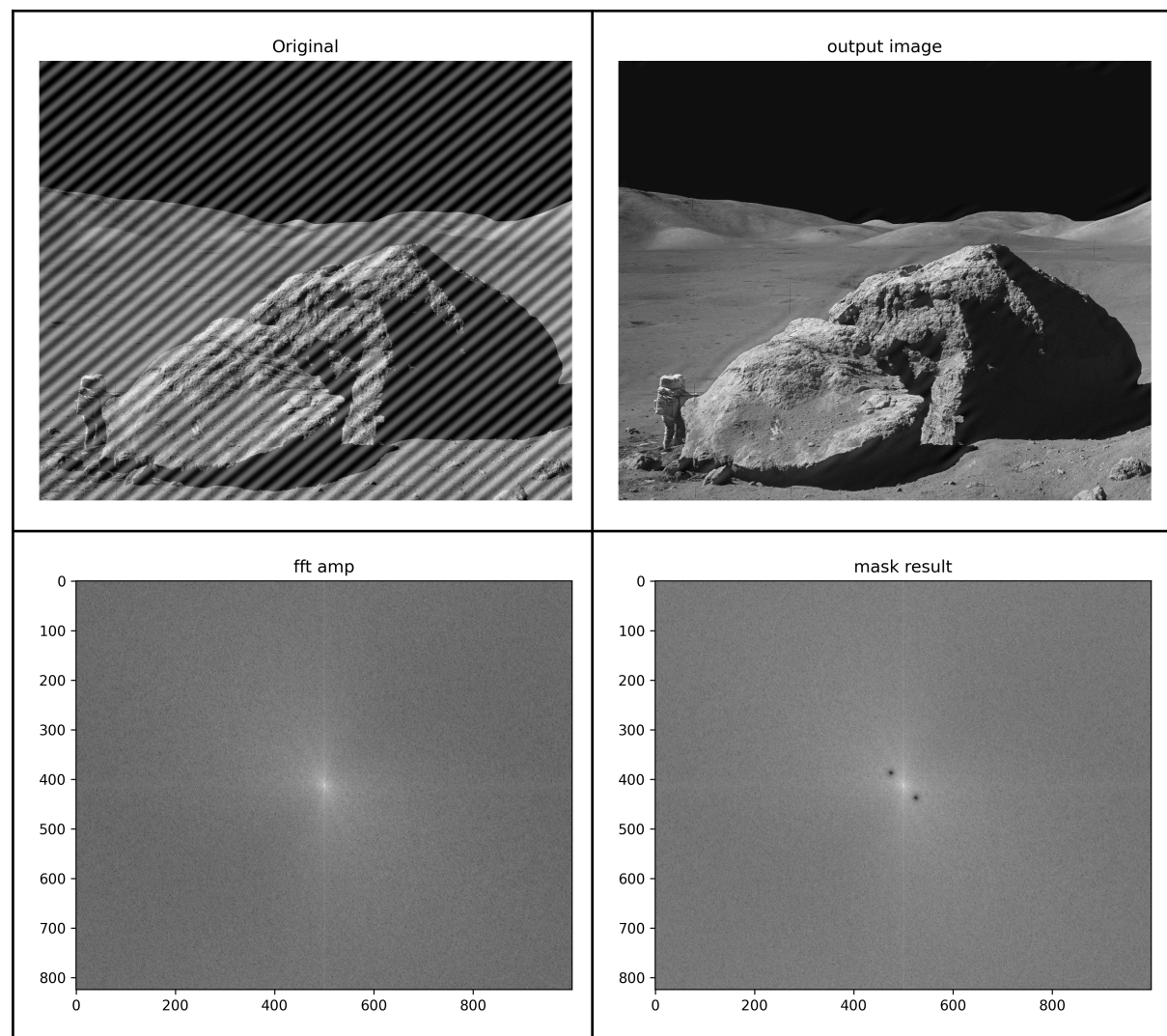
```

# 4) 輸出影像（僅顯示；檔案已在前面存好）
plt.figure()
plt.title("output image")
plt.imshow(img2_u8, cmap="gray")
plt.axis("off")
plt.tight_layout()
plt.savefig(os.path.join(out_dir, "output_image.png"), dpi=300,
bbox_inches="tight")
print("[Saved] output_image.png")

# === 直接顯示視窗 ===
if not args.no_show:
    plt.show()

if __name__ == "__main__":
    main()

```



2. Please use FFT and design a frequency filter to cancel the moire-pattern noise of the assigned image, 'car-moire-pattern.tif', and print out the source code and the processed image? (40)

```
# -*- coding: utf-8 -*-

import os
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

# ----- Utils -----
def read_gray(path: str) -> np.ndarray:
    img = Image.open(path)
    if img.mode != "L":
        img = img.convert("L")
    return np.array(img, dtype=np.float32)

def normalize_to_0_255(img: np.ndarray) -> np.ndarray:
    m, M = np.min(img), np.max(img)
    if M - m < 1e-8:
        return np.zeros_like(img, dtype=np.uint8)
    out = (img - m) / (M - m) * 255.0
    return out.astype(np.uint8)

# ----- FFT helpers -----
def fft2_shift(img: np.ndarray):
    F = np.fft.fft2(img)
    return np.fft.fftshift(F)

def ifft2_from_shift(Fs: np.ndarray) -> np.ndarray:
    F = np.fft.ifftshift(Fs)
    return np.real(np.fft.ifft2(F))

# ----- Peak detection -----
def detect_strong_peaks(logmag: np.ndarray, center_radius: int = 22,
                        k: int = 24, min_separation: int = 8):
    H, W = logmag.shape
    cy, cx = H // 2, W // 2

    yy = np.arange(H)[:, None]
    xx = np.arange(W)[None, :]
```

```

dc_mask = (yy - cy)**2 + (xx - cx)**2 <= center_radius**2
border = 3
border_mask = np.zeros_like(dc_mask, dtype=bool)
border_mask[:border, :] = True
border_mask[-border:, :] = True
border_mask[:, :border] = True
border_mask[:, -border:] = True

cand = logmag.copy()
cand[dc_mask | border_mask] = -np.inf
flat = cand.ravel()
K = min(k * 4, flat.size)
idx = np.argpartition(flat, -K)[-K:]
r, c = np.unravel_index(idx, logmag.shape)
pts = np.stack([r, c], axis=1)
vals = flat[idx]
order = np.argsort(-vals)

selected = []
for i in order:
    rr, cc = pts[i]
    ok = True
    for (r0, c0) in selected:
        if (rr - r0)**2 + (cc - c0)**2 < min_separation**2:
            ok = False
            break
    if ok:
        selected.append((int(rr), int(cc)))
    if len(selected) >= k:
        break
return selected

def build_gaussian_notch_filter(shape, peaks, sigma=8.0,
pair_enforce=True):
    H, W = shape
    yy = np.arange(H)[:, None]
    xx = np.arange(W)[None, :]
    cy, cx = H // 2, W // 2
    Hf = np.ones((H, W), dtype=np.float32)

    def notch(r0, c0):
        return 1.0 - np.exp(-((yy - r0)**2 + (xx - c0)**2) / (2.0 *
sigma**2))

```

```

used = set()
for (r, c) in peaks:
    if (r, c) in used:
        continue
    if abs(r - cy) <= 1 and abs(c - cx) <= 1:
        continue
    Hf *= notch(r, c)
    used.add((r, c))
    if pair_enforce:
        rs = (2 * cy - r) % H
        cs = (2 * cx - c) % W
        Hf *= notch(rs, cs)
        used.add((rs, cs))
return Hf

# ----- Main -----
def main():
    in_path = "car-moire-pattern.tif"
    out_path = "car_moire_filtered.png"

    if not os.path.exists(in_path):
        raise FileNotFoundError(f"找不到輸入影像: {in_path}")

    img = read_gray(in_path)
    Fs = fft2_shift(img)
    mag = np.abs(Fs)
    logmag = np.log1p(mag)

    peaks = detect_strong_peaks(logmag, center_radius=22, k=24,
min_separation=8)
    Hf = build_gaussian_notch_filter(img.shape, peaks, sigma=8.0,
pair_enforce=True)
    Ff = Fs * Hf
    rec = ifft2_from_shift(Ff)
    rec_u8 = normalize_to_0_255(rec)
    Image.fromarray(rec_u8).save(out_path)
    print(f"[Saved] result: {out_path}")
    print(f"Detected peaks = {len(peaks)}")

# ===== 儲存並顯示每張過程圖 =====

out_dir = "resault_q2"

```

```
os.makedirs(out_dir, exist_ok=True)

# 1) 原始輸入影像
plt.figure()
plt.title("Original")
plt.imshow(img.astype(np.uint8), cmap="gray")
plt.axis("off")
plt.tight_layout()
plt.savefig(os.path.join(out_dir, "original.png"), dpi=300,
bbox_inches="tight")
print("[Saved] original.png")

# 2) FFT amplitude
plt.figure()
plt.title("Log Magnitude Spectrum")
fft_vis = logmag / np.max(logmag)
plt.imshow(fft_vis, cmap="gray")
plt.axis("off")
plt.tight_layout()
plt.savefig(os.path.join(out_dir, "fft_amp.png"), dpi=300,
bbox_inches="tight")
print("[Saved] fft_amp.png")

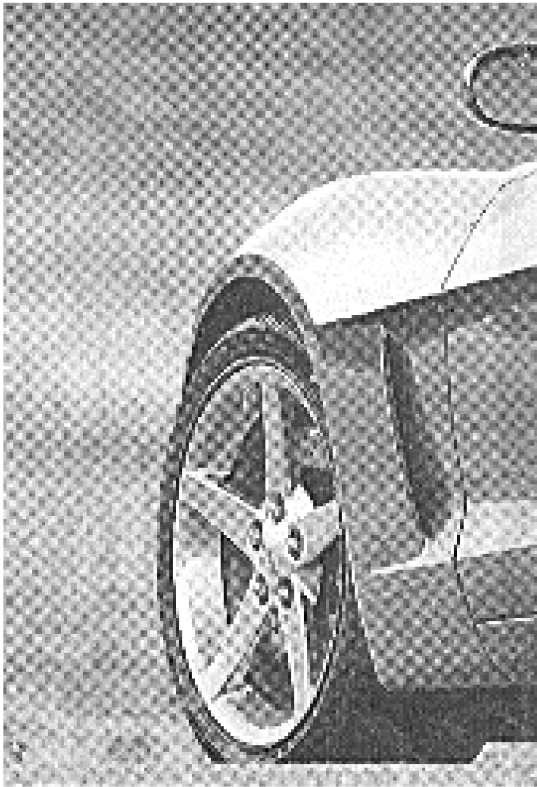
# 3) 濾波器圖像
plt.figure()
plt.title("Gaussian Notch Filter")
plt.imshow(Hf, cmap="gray")
plt.axis("off")
plt.tight_layout()
plt.savefig(os.path.join(out_dir, "mask_result.png"), dpi=300,
bbox_inches="tight")
print("[Saved] filter_mask.png")

# 4) 結果影像
plt.figure()
plt.title("Filtered (Result)")
plt.imshow(rec_u8, cmap="gray")
plt.axis("off")
plt.tight_layout()
plt.savefig(os.path.join(out_dir, "output_image.png"), dpi=300,
bbox_inches="tight")
print("[Saved] output_image.png")
```

```
plt.show()

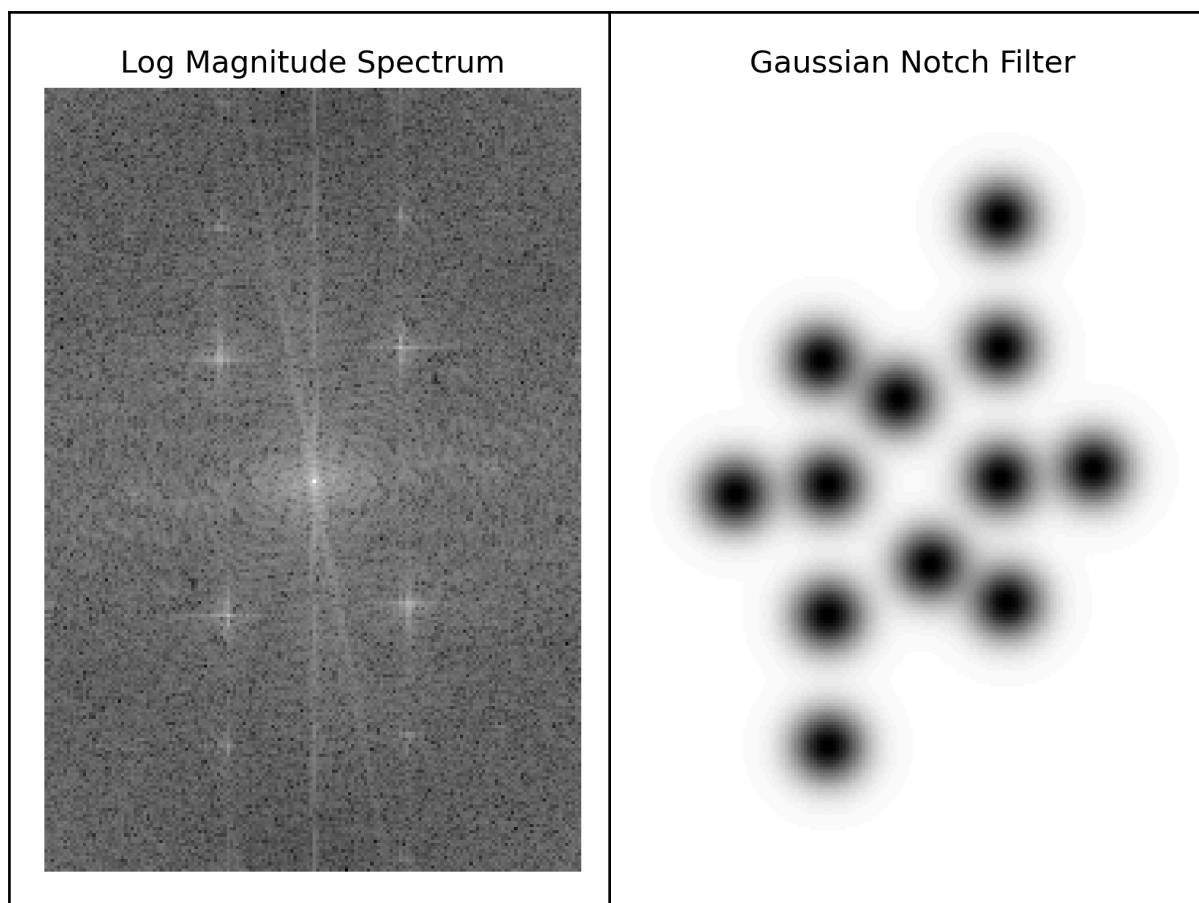
if __name__ == "__main__":
    main()
```

Original



Filtered (Result)





3. Please comment and compare your two design freq. filters? (20)

在第一題的 Astronaut 影像中，是以人工指定干擾峰位置的 Gaussian notch filter。該方法針對 2D FFT 頻譜中已知的兩個對稱亮點進行高斯凹點濾波，能有效去除單一方向的正弦條紋干擾。由於採用平滑的高斯邊界，濾波過程不會造成嚴重邊緣振盪，因此影像細節保留良好，只在明暗交界處出現輕微波紋，是頻域濾波的自然副作用。相較之下，第二題的 Car 影像則使用自動偵測多組頻率峰值的 Gaussian notch 陣列。此方法不需人工指定座標，能同時抑制多組對稱的 moiré 頻率，因此非常適合處理網格狀、周期性更複雜的干擾圖樣。

第一題的濾波器利用人工操作可保留更多原始細節，適合單一頻率干擾的情況；第二題的濾波器則能一次去除多頻干擾，但由於濾波範圍較廣，輸出影像會略顯平滑。兩者皆能有效消除頻域干擾，但設計理念不同：第一題強調針對性與細節保留，第二題強調自動化與多頻抑制能力。因此在實務應用中，若干擾位置明確，手動 Gaussian notch 效果最佳；若干擾複雜或未知，自動多峰濾波則更具效率與穩定性。